4 Model evaluation and optimization

This chapter covers

- Using cross-validation for properly evaluating the predictive performance of models
- Overfitting and how to avoid it
- Standard evaluation metrics and visualizations for binary and multiclass classification
- Standard evaluation metrics and visualizations for regression models
- Optimizing your model by selecting the optimal parameters

After you fit a machine-learning model, the next step is to assess the accuracy of that model. Before you can put a model to use, you need to know how well it's expected to predict on new data. If you determine that the predictive performance is quite good, you can be comfortable in deploying that model in production to analyze new data. Likewise, if you assess that the predictive performance isn't good enough for the task at hand, you can revisit your data and model to try to improve and optimize its accuracy. (The last section of this chapter introduces simple model optimization. Chapters 5, 7, and 9 cover more-sophisticated methods of improving the predictive accuracy of ML models.)

Properly assessing the predictive performance of an ML model is a nontrivial task. We begin this chapter by introducing statistically rigorous techniques to evaluate the predictive performance of ML models, demonstrating both pictorially and with pseudocode how to perform correct validation of a model.

From there, we dive into assessment of ML classification models, focusing on the typical evaluation metrics and graphical tools used by machine-learning practitioners. Then we introduce analogous evaluation tools for regression models. Finally, we describe a simple way to optimize the predictive performance of a model through parameter tuning.

By the end of the chapter, you'll be equipped with the means and know-how to evaluate the predictive accuracy of the ML models that you built in chapter 3 and to optimize those models for predictive accuracy (see figure 4.1). This model evaluation provides the information you need to determine whether the model you built is good enough for your use case or requires further optimization.

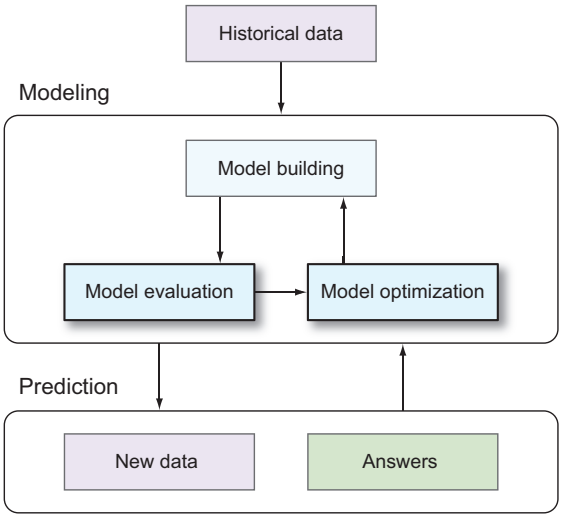


Figure 4.1 Evaluation and optimization in the ML workflow

4.1 Model generalization: assessing predictive accuracy for new data

The primary goal of supervised machine learning is accurate prediction. You want your ML model to be as accurate as possible when predicting on new data (for which the target variable is unknown). Said differently, you want your model, which has been built from training data, to generalize well to new data. That way, when you deploy the model in production, you can be assured that the predictions generated are of high quality.

Therefore, when you evaluate the performance of a model, you want to determine *how well that model will perform on new data*. This seemingly simple task is wrought with complications and pitfalls that can befuddle even the most experienced ML users.

This section describes the difficulties that arise when evaluating ML models and proposes a simple workflow to overcome those menacing issues and achieve unbiased estimates of model performance.

4.1.1 The problem: overfitting and model optimism

To describe the challenges associated with estimating the predictive accuracy of a model, it's easiest to start with an example.

Imagine that you want to predict the production of bushels of corn per acre on a farm as a function of the proportion of that farm's planting area that was treated with a new pesticide. You have training data for 100 farms for this regression problem. As you plot the target (bushels of corn per acre) versus the feature (percent of the farm treated), it's clear that an increasing, nonlinear relationship exists, and that the data also has random fluctuations (see figure 4.2).

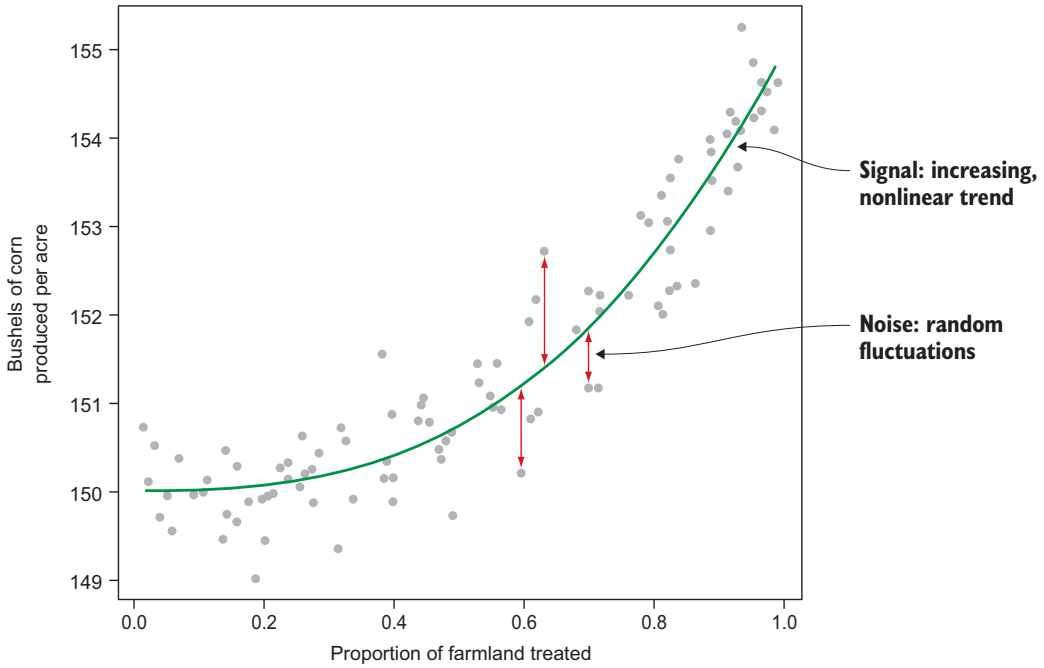


Figure 4.2 The training data for the corn production regression problem contains a clear signal and noise.

Now, suppose you want to use a simple nonparametric ML regression modeling technique to build a predictive model for corn production as a function of proportion of land treated. One of the simplest ML regression models is kernel smoothing. *Kernel smoothing* operates by taking local averages: for each new data point, the value of the target variable is modeled as the average of the target variable for only the training data whose feature value is close to the feature value of the new data point. A single parameter, called the *bandwidth parameter*, controls the size of the window for the local averaging.

Figure 4.3 demonstrates what happens for various values of the kernel-smoothing bandwidth parameters. For large values of the bandwidth, almost all of the training data is averaged together to predict the target, at each value of the input parameter. This causes the model to be flat and to underfit the obvious trend in the training data. Likewise, for small values of the bandwidth, only one or two training instances are used to determine the model output at each feature value. Therefore, the model effectively traces every bump and wiggle in the data. This susceptibility to model the intrinsic noise in the data instead of the true signal is called *overfitting*. Where you want to be is somewhere in the Goldilocks zone: not too underfit and not too overfit.

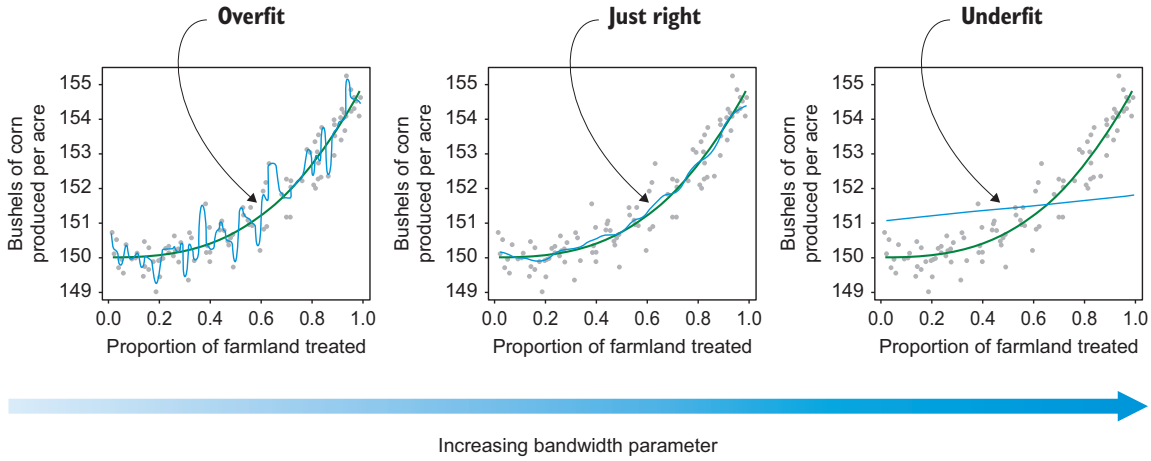


Figure 4.3 Three fits of a kernel-smoothing regression model to the corn production training set. For small values of the bandwidth parameter, the model is overfit, resulting in an overly bumpy model. For large values of the bandwidth parameter, the model is underfit, resulting in a model that's too flat. A good choice of the tuning parameter results in a fit that looks just right.

Now, let's get back to the problem at hand: determining how well your ML model will generalize to predict the corn output from data on different farms. The first step in this process is to select an evaluation metric that captures the quality of your predictions. For regression, the standard metric for evaluation is *mean squared error (MSE)*, which is the average squared difference between the true value of the target variable and the model-predicted value (later in this chapter, you'll learn about other evaluation metrics for regression and classification).

This is where things get tricky. Evaluated on the training set, the error (measured by MSE) of our model predictions gets ever smaller as the bandwidth parameter decreases. This is expected: the more flexibility that you allow the model, the better it'll do at tracing the patterns (both the signal and the noise) in the training data. But the models with smallest bandwidth are severely overfit to the training data because they trace every random fluctuation in the training set. Using these models to predict on new data will result in poor predictive accuracy, because the

new data will have its own unique random noise signatures that are different from those in the training set.

Thus, a divergence occurs between the training set error and the generalization error of an ML model. This divergence is exemplified on the corn production data in figure 4.4. For small values of the bandwidth parameter, the MSE evaluated on the training set is extremely small, whereas the MSE evaluated on new data (in this case, 10,000 new instances) is much larger. Simply put, the performance of the predictions of a model evaluated on the training set isn't indicative of the performance of that model on new data. Therefore, it's dangerous to evaluate the performance of a model on the same data that was used to train the model.

CAUTION ABOUT DOUBLE-DIPPING THE TRAINING DATA Using the training data for both model fitting and evaluation purposes can lead you to be overly optimistic about the performance of the model. This can cause you to ultimately choose a suboptimal model that performs poorly when predicting on new data.

As you see in the corn production data, choosing the model with the smallest training set MSE causes the selection of the model with the smallest bandwidth. On the training

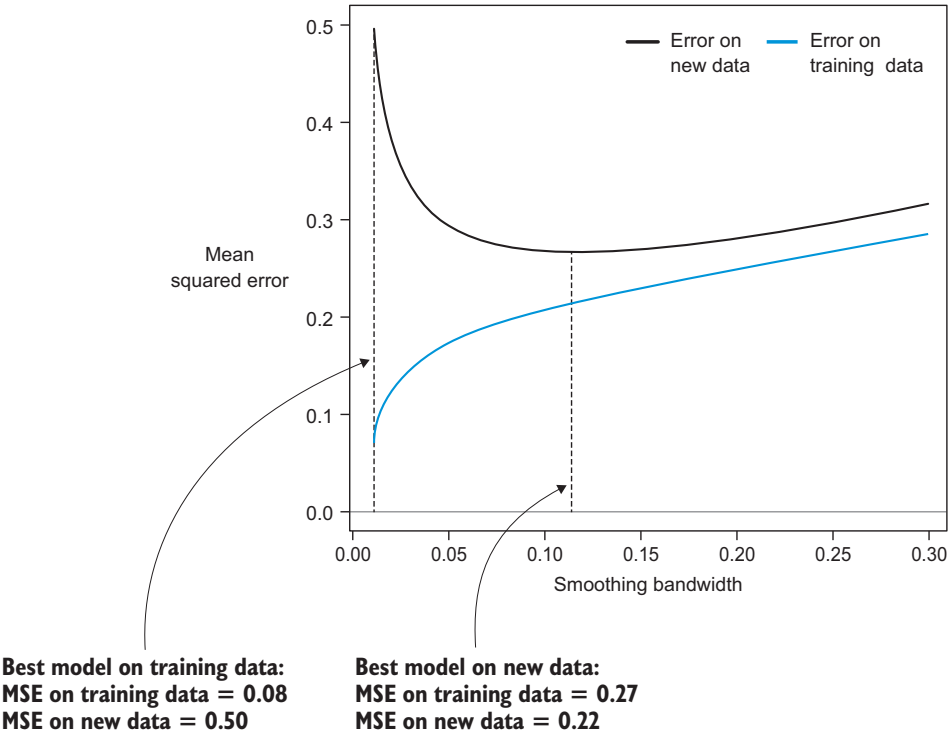


Figure 4.4 Comparison of the training set error to the error on new data for the corn production regression problem. The training set error is an overly optimistic measure of the performance of the model for new data, particularly for small values of the bandwidth parameter. Using the training set error as a surrogate for the prediction error on new data will get you into a lot of trouble.

set, this model yields an MSE of 0.08. But when applied to new data, the same model yields an MSE of 0.50, which is much worse than the optimal model (bandwidth = 0.12 and MSE = 0.27).

You need an evaluation metric that better approximates the performance of the model on new data. This way, you can be confident about the accuracy of your model when deployed to make predictions on new data. This is the topic of the next subsection.

4.1.2 The solution: cross-validation

We've diagnosed the challenge in model evaluation: the training set error isn't indicative of the model error when applied to new data. To get a good estimate of what your error rate will be for new data, you must use a more sophisticated methodology called *cross-validation* (often abbreviated *CV*) that rigorously employs the training set to evaluate what the accuracy will be on new data.

The two most commonly used methods for cross-validation are the holdout method and k-fold cross-validation.

THE HOLDOUT METHOD

Using the same training data to both fit and evaluate the accuracy of a model produces accuracy metrics that are overly optimistic. The easiest way around this is to use separate training and testing subsets. You use only the training subset to fit the model, and only the testing subset to evaluate the accuracy of the model.

This approach is referred to as the *holdout method*, because a random subset of the training data is held out from the training process. Practitioners typically leave out 20–40% of the data as the testing subset. Figure 4.5 depicts the basic algorithmic flow of the holdout method, and listing 4.1 provides the Python pseudocode.

Listing 4.1 Cross-validation with the holdout method

```
# assume that we begin with two inputs:
#    features - a matrix of input features
#    target - an array of target variables corresponding to those features
features = rand(100,5)
target = rand(100) > 0.5

N = features.shape[0] # The total number of instances
N_train = floor(0.7 * N) # The total number of training instances

idx = random.permutation(N)                    <-- Randomizes index

idx_train = idx[:N_train]
idx_test = idx[N_train:]                       Splits index

features_train = features[idx_train,:]
target_train = target[idx_train]
features_test = features[idx_test,:]
target_test = target[idx_test]                 Breaks your data into training and testing subsets
```

```
# Build, predict, evaluate (to be filled out)
# model = train(features_train, target_train)
# preds_test = predict(model, features_test)
# accuracy = evaluate_acc(preds_test, target_test)
```

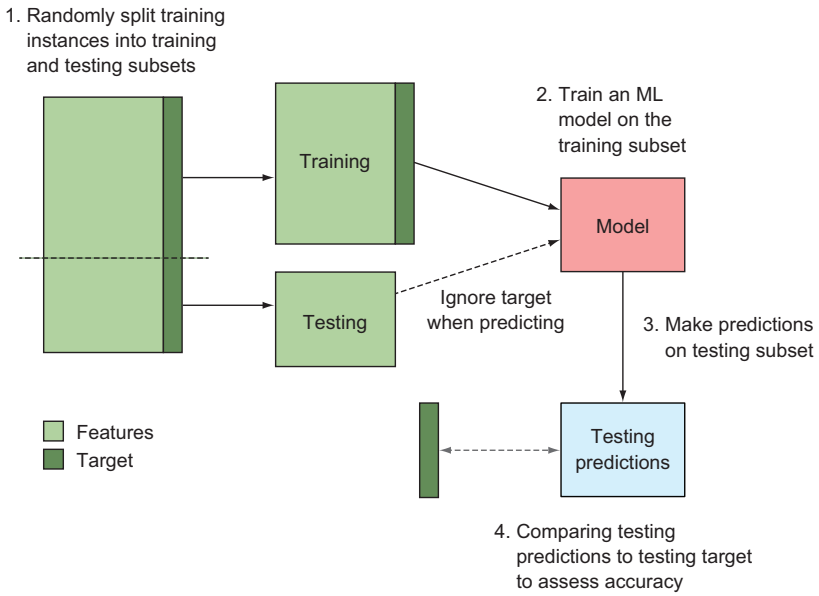


Figure 4.5 Flowchart of the holdout method of cross-validation.

Now, let's apply the holdout method to the corn production data. For each value of the bandwidth parameter, you apply the holdout method (using a 70/30 split) and compute the MSE on the predictions for the held-out 30% of data. Figure 4.6 demonstrates how the holdout method estimates of the MSE stack up to the MSE of the model when applied to new data. Two main things stand out:

- The error estimates computed by the holdout method are close to the new-data error of the model. They're certainly much closer than the training set error estimates (figure 4.4), particularly for small-bandwidth values.
- The holdout error estimates are noisy. They bounce around wildly compared to the smooth curve that represents the error on new data.

You could beat down the noise by doing repeated random training-testing splits and averaging the result. But over multiple iterations, each data point will be assigned to the testing set a different number of times, which could bias the result.

A better approach is to do k-fold cross-validation.

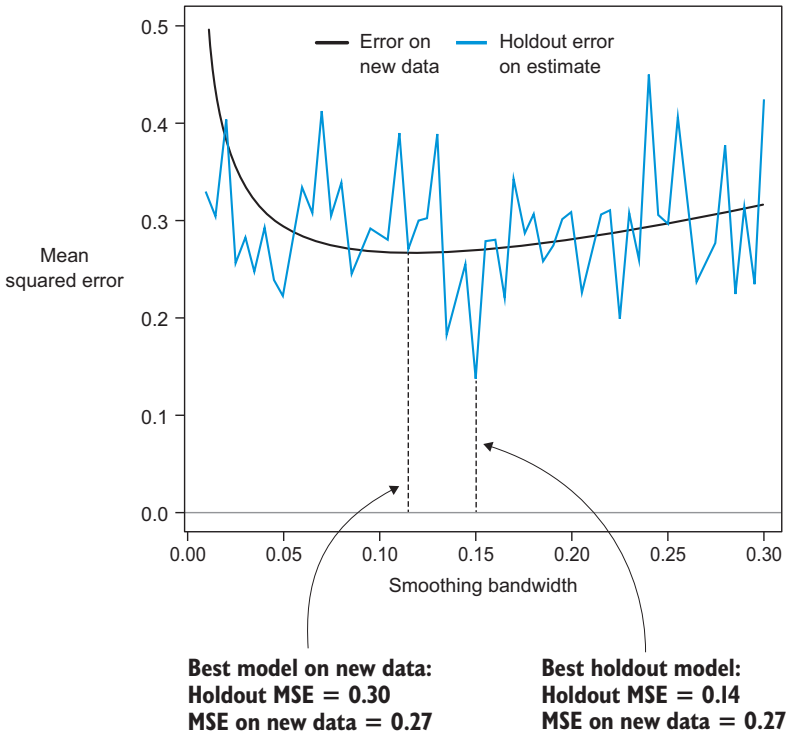


Figure 4.6 Comparison of the holdout error MSE to the MSE on new data, using the corn production dataset. The holdout error is an unbiased estimate of the error of each model on new data. But it's a noisy estimator that fluctuates wildly between 0.14 and 0.40 for bandwidths in the neighborhood of the optimal model (bandwidth = 0.12).

K-FOLD CROSS-VALIDATION

A better but more computationally intensive approach to cross-validation is *k-fold cross-validation*. Like the holdout method, k-fold cross-validation relies on quarantining subsets of the training data during the learning process. The primary difference is that k-fold CV begins by randomly splitting the data into k disjoint subsets, called *folds* (typical choices for k are 5, 10, or 20). For each fold, a model is trained on all the data *except* the data from that fold and is subsequently used to generate predictions for the data from that fold.

After all k-folds are cycled through, the predictions for each fold are aggregated and compared to the true target variable to assess accuracy. Figure 4.7 illustrates k-fold cross-validation, and listing 4.2 provides the pseudocode.

Finally, let's apply k-fold cross-validation to the corn production data. For each value of the bandwidth parameter, you apply k-fold cross-validation with k = 10 and compute the cross-validated MSE on the predictions. Figure 4.8 demonstrates how the k-fold cross-validation MSE estimates stack up to the MSE of the model when applied

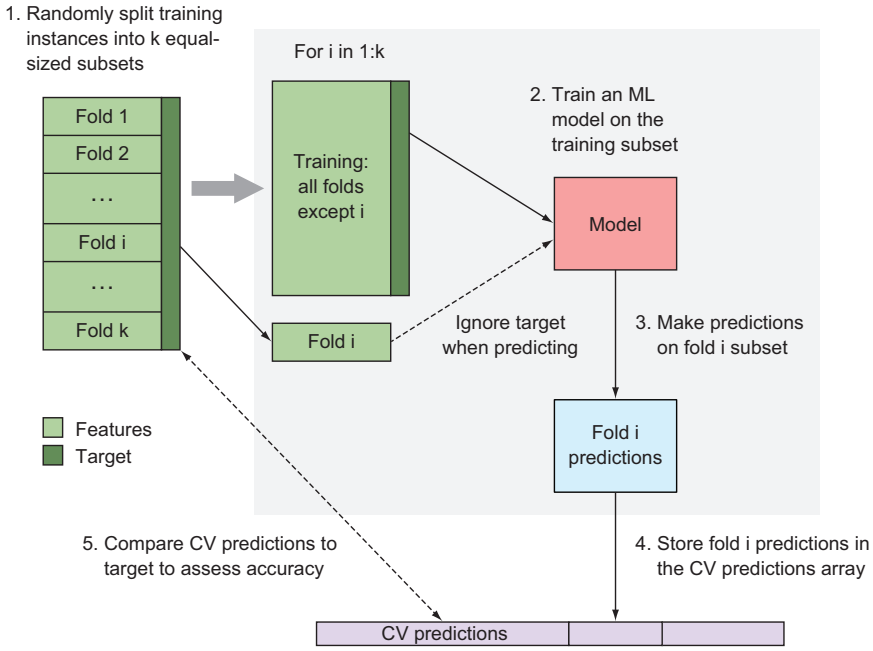


Figure 4.7 Flowchart of k-fold cross-validation

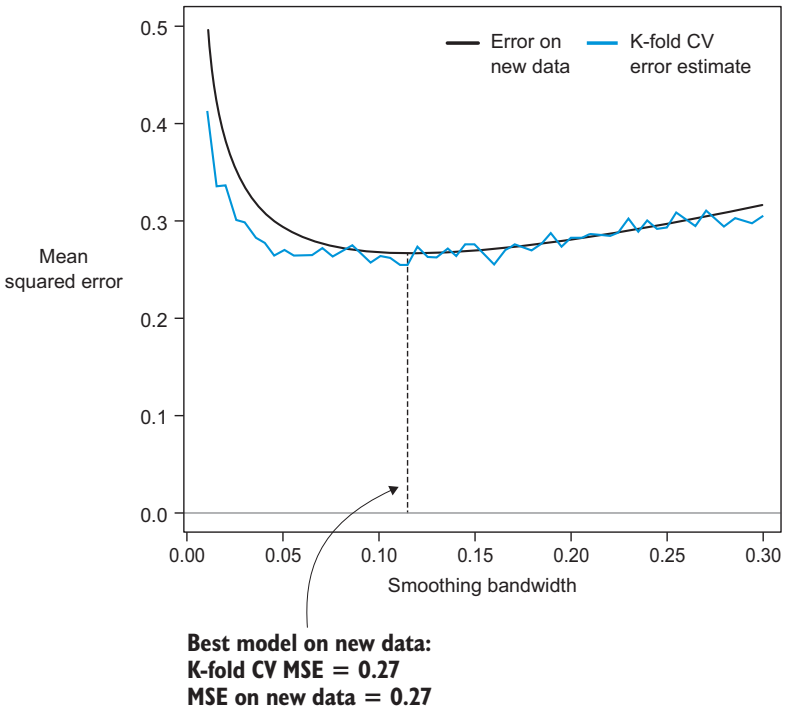


Figure 4.8 Comparison of the k-fold cross-validation error MSE to the MSE on new data, using the corn production dataset. The k-fold CV error is a good estimate for how the model will perform on new data, allowing you to use it confidently to forecast the error of the model and to select the best model.

to new data. Clearly, the k-fold cross-validation error estimate is close to the error of the model on future data.

Listing 4.2 Cross-validation with k-fold cross-validation

```
N = features.shape[0]
K = 10 # number of folds

preds_kfold = np.empty(N)
folds = np.random.randint(0, K, size=N)

for idx in np.arange(K):

    features_train = features[folds != idx,:]
    target_train = target[folds != idx]
    features_test = features[folds == idx,:]

    # Build and predict for CV fold (to be filled out)
    # model = train(features_train, target_train)
    # preds_kfold[folds == idx] = predict(model, features_test)

# accuracy = evaluate_acc(preds_kfold, target)
```

Loops over the folds

Breaks your data into training and testing subsets

4.1.3 Some things to look out for when using cross-validation

Cross-validation gives you a way to estimate how accurately your ML models will predict when deployed in the wild. This is extremely powerful, because it enables you to select the best model for your task.

But when you apply cross-validation to real-world data, you need to watch out for a few things:

- Cross-validation methods (including both the holdout and k-fold methods) assume that the training data forms a representative sample from the population of interest. If you plan to deploy the model to predict on new data, that data should be well represented by the training data. If not, the cross-validation error estimates may be overly optimistic for the error rates on future data. Solution: Ensure that any potential biases in the training data are addressed and minimized.
- Some datasets use features that are temporal—for instance, using last month's revenue to forecast this month's revenue. If this is the case with your data, you must ensure that features that are available in the future can never be used to predict the past. Solution: You can structure your cross-validation holdout set or k-folds so that all the training set data is collected previous to the testing set.
- The larger the number of folds used in k-fold cross-validation, the better the error estimates will be, but the longer your program will take to run. Solution: Use at least 10 folds (or more) when you can. For models that train and predict quickly, you can use leave-one-out cross-validation (k = number of data instances).

Next, you'll build off of these cross-validation tools and take a deeper look at how to perform rigorous model evaluation for classification models.

4.2 Evaluation of classification models

We begin our discussion of evaluating classification models by presenting problems with only two classes, also known as *binary classification*. Chapter 3 introduced binary classification in machine learning as a powerful method for predicting a positive/negative outcome based on many factors or variables. A good example of binary classification is the detection of diseases or survival predictions.

Imagine that you want to predict whether a Titanic passenger would survive, based on personal, social, and economic factors. You'd gather everything you know about the passengers and train a classifier that could relate all this information to their survival probability. You first saw this example in chapter 2, but the first five rows of the Titanic Passengers dataset is shown again in figure 4.9.

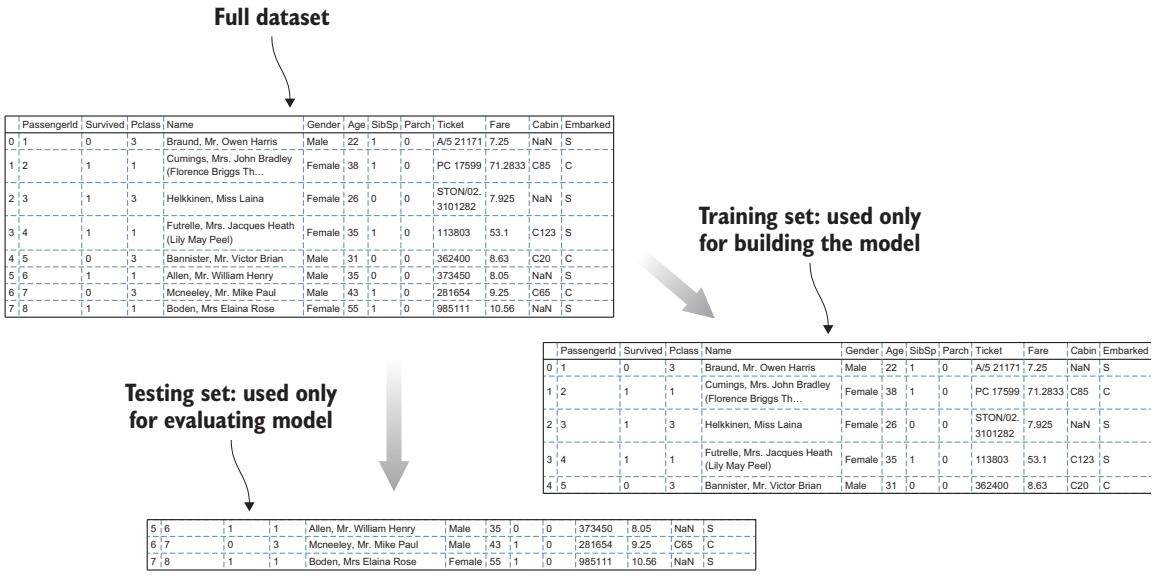
Target column

	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	Male	22	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th…	Female	38	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Helkkinen, Miss Laina	Female	26	0	0	STON/02. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	Female	35	1	0	113803	53.1	C123	S
4	5	0	3	Allen, Mr. William Henry	Male	35	0	0	373450	8.05	NaN	S

Figure 4.9 The first five rows of the Titanic Passengers dataset. The target column indicates whether a passenger survived the sinking of the ship or died.

To build your classifier, you feed this dataset into a classification algorithm. Because the dataset consists of different types of data, you have to make sure the algorithm knows how to deal with these types. As discussed in the previous chapters, you might need to process the data prior to training the model, but for this chapter you'll view the classifier as a black box that has learned the mapping from the input variables to the target variable. The goal of this section is to evaluate the model in order to optimize the prediction accuracy and compare with other models.

With the data ready, you move to the next task: cross-validation. You'll divide the full dataset into training and testing sets and use the holdout method of cross-validation. The model will be built on a training set and evaluated on a held-out testing set. It's important to reiterate that your goal isn't necessarily to obtain the maximum model accuracy on the training data, but to obtain the highest predictive accuracy on unseen data. In the model-building phase, you're not yet in possession of this data, by definition, so you pretend that some of the training data is hidden for the learning algorithm.



	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	Male	22	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th…	Female	38	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Helkkinen, Miss Laina	Female	26	0	0	STON/02. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	Female	35	1	0	113803	53.1	C123	S
4	5	0	3	Bannister, Mr. Victor Brian	Male	31	0	0	362400	8.63	C20	C
5	6	1	1	Allen, Mr. William Henry	Male	35	0	0	373450	8.05	NaN	S
6	7	0	3	Mcneeley, Mr. Mike Paul	Male	43	1	0	281654	9.25	C65	C
7	8	1	1	Boden, Mrs Elaina Rose	Female	55	1	0	985111	10.56	NaN	S

	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	Male	22	1	0	A/5 21171	7.25	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th…	Female	38	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Helkkinen, Miss Laina	Female	26	0	0	STON/02. 3101282	7.925	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	Female	35	1	0	113803	53.1	C123	S
4	5	0	3	Bannister, Mr. Victor Brian	Male	31	0	0	362400	8.63	C20	C

5	6	1	1	Allen, Mr. William Henry	Male	35	0	0	373450	8.05	NaN	S
6	7	0	3	Mcneeley, Mr. Mike Paul	Male	43	1	0	281654	9.25	C65	C
7	8	1	1	Boden, Mrs Elaina Rose	Female	55	1	0	985111	10.56	NaN	S

Figure 4.10 Splitting the full dataset into training and testing sets allows you to evaluate the model.

Figure 4.10 illustrates the dataset-splitting step in this particular example.

With the training set ready, you can build the classifier and make predictions on the testing set. Following the holdout method of figure 4.5, you obtain a list of prediction values: 0 (died) or 1 (survived) for all rows in the test set. You then go to step 3 of the evaluation workflow and compare these predictions to the actual survival values to obtain a performance metric that you can optimize.

The simplest performance measure of a classification model is to calculate the fraction of *correct* answers; if three out of four rows were correctly predicted, you'd say the *accuracy* of the model on this particular validation set is 3/4 = 0.75, or 75%. Figure 4.11 illustrates this result. The following sections introduce more-sophisticated ways of performing this comparison.

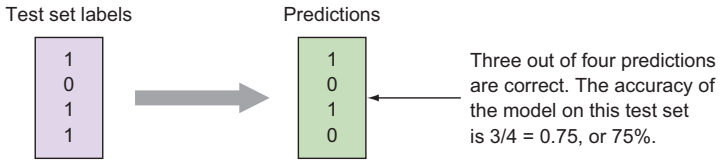


Figure 4.11 Comparing the testing set predictions with the actual values gives you the accuracy of the model.

4.2.1 Class-wise accuracy and the confusion matrix

The predictions provide more information than simply being correct or not. For example, you can analyze the accuracy per class (how many were predicted to survive but actually died or survived). For binary classification, you can be wrong in two ways: predicting 0 when the correct value is 1, or predicting 1 when the correct value is 0. In the same way, you can be correct in two ways. Figure 4.12 illustrates.

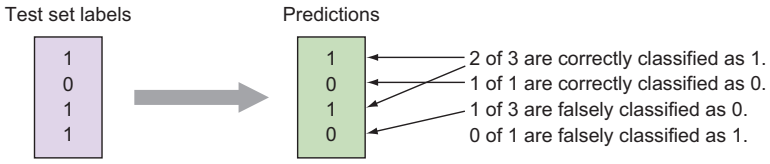


Figure 4.12 Counting the class-wise accuracy and error rate gives you more information on the model accuracy.

In many classification problems, it's useful to go beyond the simple counting accuracy and look at this class-wise accuracy, or class confusion. It turns out to be useful to display these four numbers in a two-by-two diagram called a *confusion matrix*, shown in figure 4.13.

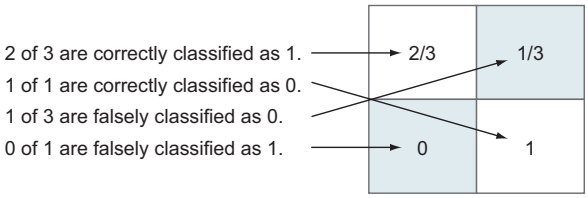


Figure 4.13 Organizing the class-wise accuracy into a confusion matrix

Each element in the matrix shows the class-wise accuracy or confusion between the positive and the negative class. Figure 4.14 relates the specific confusion matrix in figure 4.13 to the general concept of receiver operating characteristics (ROCs) that you'll employ widely throughout the rest of this book. Although these terms can be a bit confusing at first, they'll become important when talking to other people about the performance of your model.

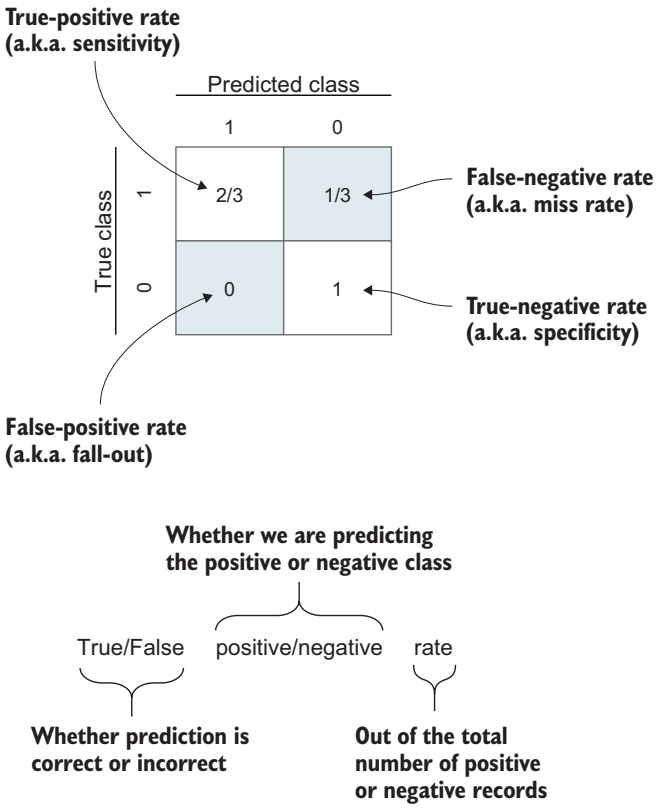
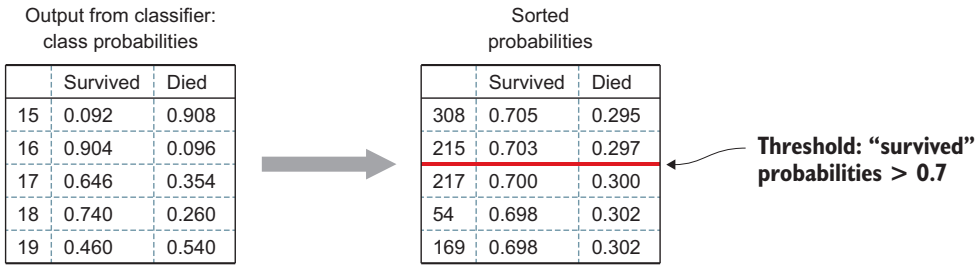


Figure 4.14 The confusion matrix for your binary classifier tested on only four rows. The ROC metrics pointed out in the figure are chopped up and explained in the bottom box.

4.2.2 Accuracy trade-offs and ROC curves

So far you've looked only at predictions for which the output is the predicted class; in our Titanic example, 1 for survival and 0 otherwise. Machine-learning predictions usually hold a degree of uncertainty, and many classification algorithms output not only the zero-one predictions, but the full prediction *probabilities*. For example, what was simply predicted as *survived* in our Titanic model may have had a probability of survival of 0.8, 0.99, or 0.5. It's clear that there's a big difference in the confidence of these answers, and in this section you'll take advantage of this information to evaluate your models in more detail.

The output of a *probabilistic classifier* is what we call the *probability vectors* or *class probabilities*. For every row in the test set, you get a real-valued number from 0 to 1 for every class in the classifier (summing to 1). Until now, you've made predictions by considering probabilities above 0.5 to determine the class predictions, from which you calculated all the performance metrics from the previous section. We say that the *threshold* that determines the class is 0.5. It's clear that you could choose any other threshold and would get different values for all of your metrics.



	Survived	Died
15	0.092	0.908
16	0.904	0.096
17	0.646	0.354
18	0.740	0.260
19	0.460	0.540

	Survived	Died
308	0.705	0.295
215	0.703	0.297
217	0.700	0.300
54	0.698	0.302
169	0.698	0.302

Figure 4.15 A subset of probabilistic predictions from the Titanic test set. After sorting the full table by decreasing survival probability, you can set a threshold and consider all rows above this threshold as survived. Note that the indices are maintained so you know which original row the instance refers to.

Figure 4.15 shows the process of sorting the probability vectors and setting a threshold of 0.7. All rows above the line are now predicted to survive, and you can compare these to the actual labels to get the confusion matrix and the ROC metrics at this particular threshold. If you follow this process for all thresholds from 0 to 1, you define the *ROC curve*, shown in figure 4.16.

From figure 4.16, you can read out the confusion matrix at all thresholds, making the ROC curve a powerful visualization tool when you're evaluating classifier performance. Given the true and predicted labels from any cross-validation process, the ROC curve is calculated as shown in listing 4.3.

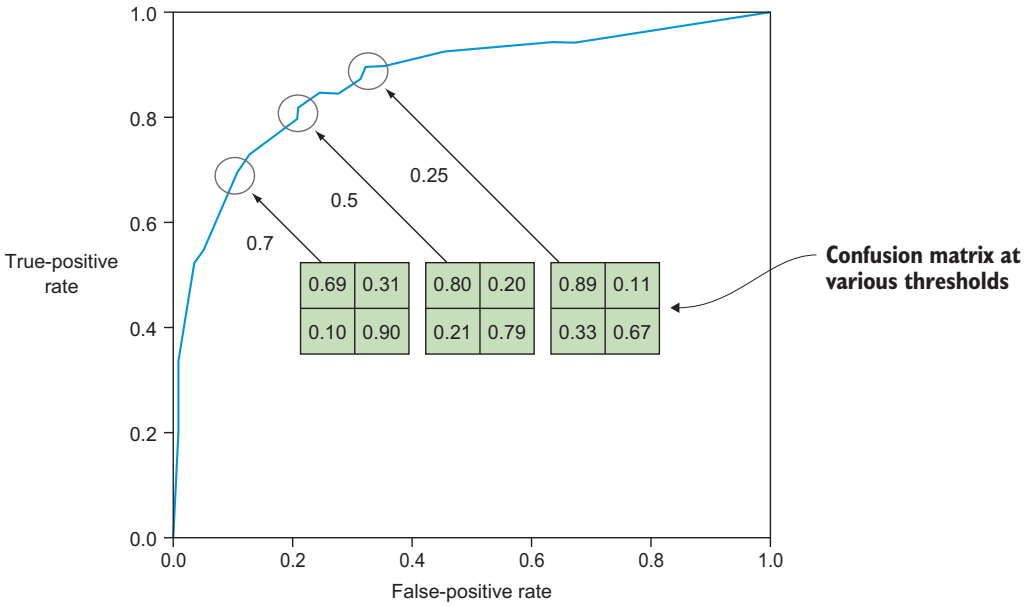


Figure 4.16 The ROC curve defined by calculating the confusion matrix and ROC metrics at 100 threshold points from 0 to 1. By convention, you plot the false-positive rate on the x-axis and the true-positive rate on the y-axis.

Listing 4.3 The ROC curve

```
import numpy as np

def roc_curve(true_labels, predicted_probs, n_points=100, pos_class=1):

  thr = np.linspace(0,1,n_points)
  tpr = np.zeros(n_points)
  fpr = np.zeros(n_points)

  pos = true_labels == pos_class
  neg = np.logical_not(pos)
  n_pos = np.count_nonzero(pos)
  n_neg = np.count_nonzero(neg)

  for i,t in enumerate(thr):
    tpr[i] = np.count_nonzero(np.logical_and(
             predicted_probs >= t, pos)) / n_pos
    fpr[i] = np.count_nonzero(np.logical_and(
             predicted_probs >= t, neg)) / n_neg
  return fpr, tpr, thr
```

Returns the false-positive and true-positive rates at n_points thresholds for the given true and predicted labels

Allocates the threshold and ROC lists

Precalculates values for the positive and negative cases, used in the loop

For each threshold, calculates the rate of true and false positives

If you follow the ROC curve, you see that when the false-positive rate increases, the true-positive rate decreases. This trade-off is the "no free lunch" of machine learning because you're able to sacrifice the fraction of instances that you classify correctly for more certainty that you're correct, and vice versa, depending on your choice of the probability threshold parameter.

In real-world scenarios, this trade-off can be extremely important to evaluate. If you're classifying whether a patient has cancer or not, it's much better to classify a few extra healthy patients as sick, and avoid classifying any sick patients as healthy. So you'd select the threshold that would minimize the false-negative rate, and hence maximize the true-positive rate and place you as far as possible in the top of the ROC plot while sacrificing the false-positive rate.

Another good example is spam filters, where you'll need to choose between unwanted emails being shown in your inbox or wanted emails being dumped in the spam folder. Or credit-card companies detecting fraudulent activities—would you rather call your customers often with false alarms or risk missing potential fraudulent transactions?

In addition to the trade-off information, the ROC curve itself also provides a view of the overall performance of the classifier. A perfect classifier would have no false positives and no missed detections, so the curve would be pushed to the top-left corner, as illustrated in figure 4.17. This leads us naturally to another evaluation metric: the area under the ROC curve (AUC). The larger this area, the better the classification performance. The AUC is a widely used choice for evaluating and comparing models, although in most cases it's important to inspect the full ROC curve in order to

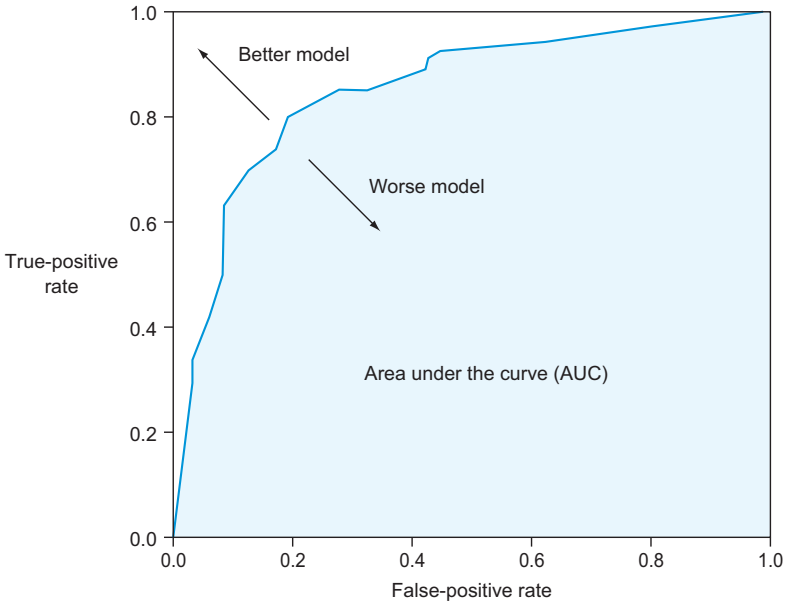


Figure 4.17 The ROC curve illustrates the overall model performance. You can quantify this by defining the AUC metric: the area under the ROC curve.

understand the performance trade-offs. You'll use the ROC curve and the AUC evaluation metric to validate classification models throughout the rest of this book.

Listing 4.4 The area under the ROC curve

```
from numpy import trapz

def auc(true_labels, predicted_labels, pos_class=1):
    fpr, tpr, thr = roc_curve(true_labels, predicted_labels,
                              pos_class=pos_class)
    area = -trapz(tpr, x=fpr)
    return area
```

Returns the area under the ROC curve, given the true labels and the corresponding labels predicted by a classification model

The false-positive and true-positive rate of the ROC curve from listing 4.3

Area integral is calculated using the trapezoidal method available in numpy

4.2.3 Multiclass classification

So far you've looked only at binary, or two-class, classification problems, but luckily you can use many of the same tools for multiclass classifiers. A well-known multiclass classification problem is that of handwritten digit recognition. We all send physical mail from time to time, and there's a good chance that a machine-learning algorithm has been used in the process of determining the endpoint address of your letter. That sounds like a huge challenge if your handwriting is anything like ours, but such automated systems have nevertheless been in use by postal services for decades.

Because of the early success of machine learning on handwritten digit recognition, this example has been used throughout the ML literature as a benchmark of multiclass classification performance. The idea is to scan the handwritten digits and divide them into images with one letter in each. You then use image-processing algorithms or build a multiclass classifier on the raw grayscale pixels that can predict the digit. Figure 4.18 shows a few examples of the handwritten digit dataset known as MNIST.

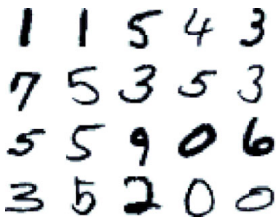


Figure 4.18 Handwritten digits in the MNIST dataset. The entire dataset consists of 80,000 such digits, each in a 28 x 28–pixel image. Without any image processing, each row of our dataset then consists of a known label (0 to 9) and 784 features (one for each of the 28 x 28 pixels).

You use the random forest algorithm (introduced in chapter 3) to build a classifier from the training set, and you generate the confusion matrix from the held-out testing set. Remember that you've worked with the confusion matrix for only binary classification. Luckily, you can easily define it for multiple classes, as every element in the matrix is the class on the row versus the class on the column. For the MNIST classifier, you can see in figure 4.19 that most of the power is located on the matrix diagonal, as

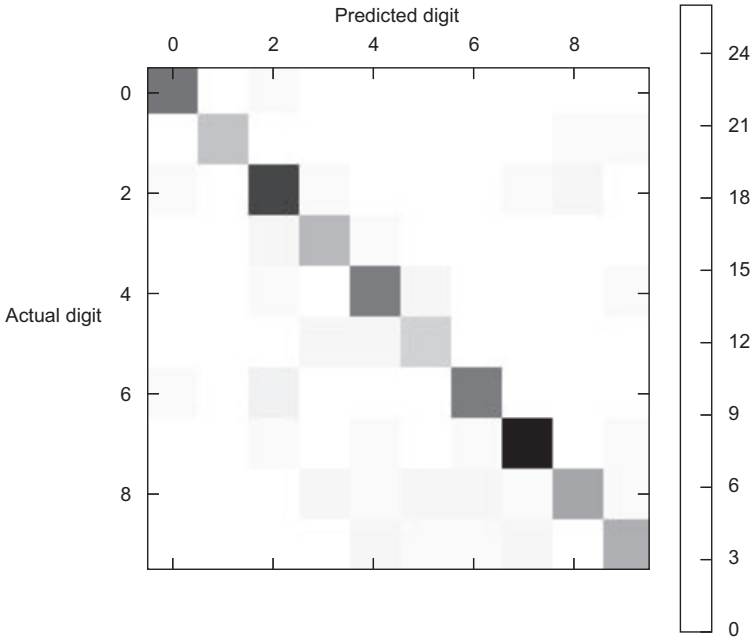


Figure 4.19 The confusion matrix for the 10-class MNIST handwritten digit classification problem

it should be, because it shows the number of instances that are *correctly* classified for each digit. The largest nondiagonal items show where the classifier is most confused. Inspecting the figure, you can see that the greatest confusion occurs between digits 4 and 9, 3 and 5, and 7 and 9, which makes sense, given what you know about the shape of the digits.

The reason for displaying the class-wise accuracy in the form of a matrix is to take advantage of our excellent visual abilities to process more information. In figure 4.19, you can clearly see how applying contrast to the confusion matrix can help take advantage of this ability.

So how do you generate the ROC curve for multiclass classifiers? The ROC curve is in principle applicable to only binary classification problems, because you divide the predictions into *positive* and *negative* classes in order to get ROC metrics such as the true-positive rate and false-positive rate commonly used on the ROC curve axis. To simulate binary classification in a multiclass problem, you use the *one-versus-all* trick. For each class, you denote the particular class as the *positive* class, and everything else as the *negative* class, and you draw the ROC curve as usual. The 10 ROC curves from running this process on the MNIST classifier are shown in figure 4.20. The most accurately classified digits are 0 and 1, consistent with the confusion matrix in figure 4.19. The confusion matrix, however, is generated from the most

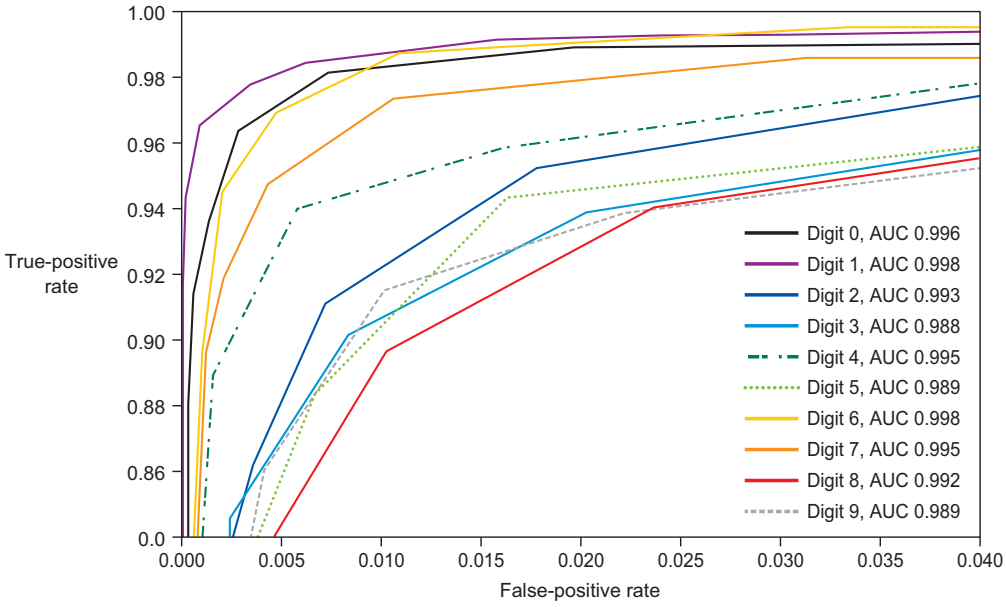


Figure 4.20 The ROC curves for each class of the MNIST 10-class classifier using the one-versus-all method for simulating a binary classification problem. Note that because the classifier is so good, we've zoomed closely into the top corner of the ROC curve in order to see any differences in the model performance between the classes. The AUC is calculated for each class and also shows a well-performing model overall.

probable class predictions, whereas the ROC curve shows the performance of the class at all probability thresholds.

Keep in mind, however, that the multiclass ROC curves don't show the entire confusion matrix on the curve. In principle, there's a full 10 x 10 confusion matrix at every point on the ROC curve, but we can't visualize this in a sufficiently simple way. In the multiclass case, it's therefore important to look at both the confusion matrix and the ROC curve.

4.3 Evaluation of regression models

You've already looked at regression models in previous chapters. Generally, *regression* is the term you use for models that predict a numeric outcome, such as an integer or floating-point value. For regression, you use a different set of performance metrics that we introduce in this section.

You'll use the Auto MPG dataset, first introduced in chapter 2, as the working example in this section. Figure 4.21 shows a small subset of this dataset. You run this dataset through all the necessary data transformations (see section 2.2 for more information about data transformations) and choose an appropriate model as discussed in chapters 2 and 3. In this case, you're interested in measuring the model performance.

Target variable

	MPG	Cylinders	Displacement	Horsepower	Weight	Acceleration	Model/year	Origin
0	18	8	307	130	3504	12.0	70	1
1	15	8	350	165	3693	11.5	70	1
2	18	8	318	150	3436	11.0	70	1
3	16	8	304	150	3433	12.0	70	1
4	17	8	302	140	3449	10.5	70	1

Figure 4.21 A subset of the Auto MPG dataset

Using the basic model-evaluation workflow introduced at the beginning of this chapter, you use the data and your choice of algorithm to build a cross-validated regression model. Potential model performance metrics used in this process are introduced in the following sections, but figure 4.22 shows the most basic visualization of the regression performance on which those metrics are based: the scatter plot of predicted versus actual values.

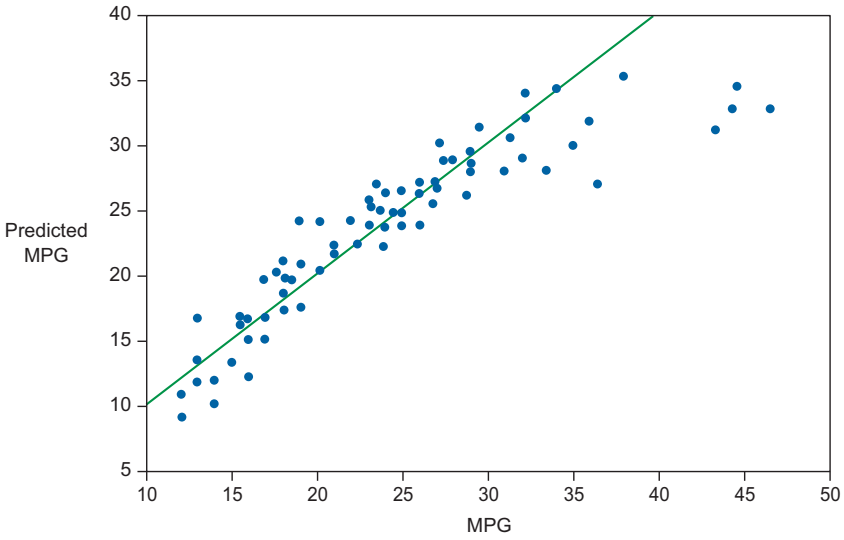


Figure 4.22 Scatter plot of the predicted MPG versus actual values from the testing set. The diagonal line shows the optimal model.

4.3.1 Using simple regression performance metrics

In contrast to classification models, regression carries no simple notion of a *correct* prediction. A numeric prediction is in general unlikely to be exactly right, but it can be *close to* or *far from* the correct value. This is also a consequence of the nature of what it means to be a *correct* value, because we usually consider numerical measurements to be drawn from a distribution with a degree of uncertainty known as the *error*. This section introduces two simple metrics to measure the regression performance: the root-mean-square error (the square root of the MSE) and the R-squared value.

The simplest form of performance measurement of a regression model is the *root-mean-square error*, or *RMSE*. This estimator looks at the difference from each of the predicted values to the known values, and calculates the mean in a way that's immune to the fact that predicted values can be both higher and lower than the actual values. Figure 4.23 illustrates RMSE calculation.

To encourage a better understanding of the details in the RMSE calculation, the following listing shows a code snippet.

Listing 4.5 The root-mean-square error

```
def rmse(true_values, predicted_values):
  n = len(true_values)
  residuals = 0
  for i in range(n):
    residuals += (true_values[i] - predicted_values[i])**2.
  return np.sqrt(residuals/n)
```

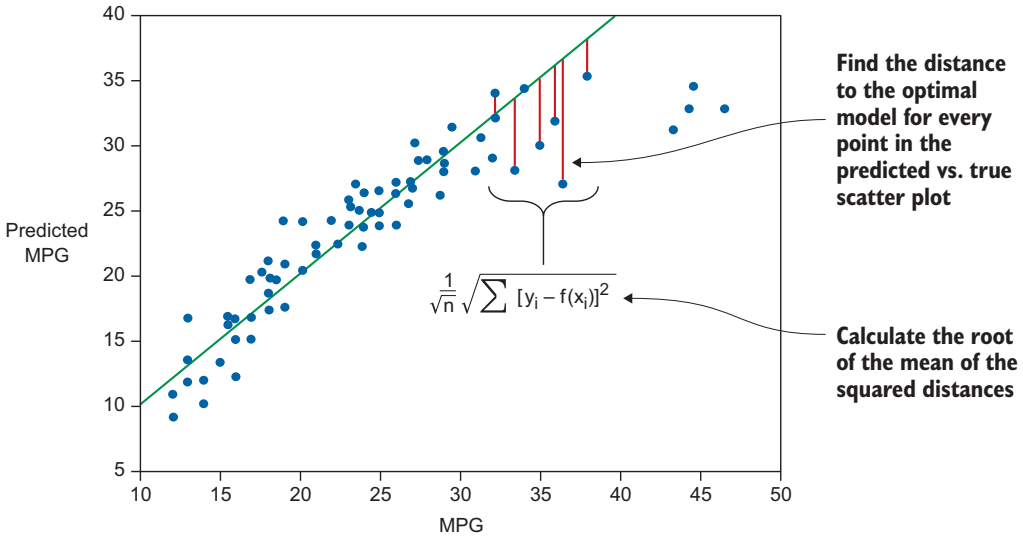


Figure 4.23 An RMSE calculation: in the equation, y_i and x_i are the ith target and feature vector, respectively, and f(x) denotes the application of the model to the feature vector, returning the predicted target value.

The advantage of RMSE is that the result is in the same units as the values themselves, but it's also a disadvantage in the sense that the RMSE value depends on the scale of the problem, and thus isn't easily comparable across datasets. If the predicted or actual values are larger numbers, the RMSE will be correspondingly higher. Although this isn't a problem when comparing models in the same project, it can be a challenge to understand the overall model performance and compare it to other models in general.

To overcome this, often it's worthwhile to also compute the R-squared, or R^2, metric, whose response is relative and always in the 0–1 range. If the model can predict the data better, the R-squared value is closer to 1. The following listing shows more details of the R-squared calculation.

Listing 4.6 The R-squared calculation

```
def r2(true_values, predicted_values):
  n = len(true_values)
  mean = np.mean(true_values)
  residuals = 0
  total = 0
  for i in range(n):
    residuals += (true_values[i] - predicted_values[i])**2.
    total += (true_values[i] - mean)**2.
  return 1.0 - residuals/total
```

Whether using MSE, RMSE, or R^2 as the evaluation metric, you should always keep the following in mind:

- Always use cross-validation to assess the model. If you don't, the metrics will always improve with increasing model complexity, causing overfitting.
- Wherever possible, the evaluation metric should align with the problem at hand. For instance, if predicting MPG from automobile features, an RMSE of 5 means that you expect the average prediction to differ from the true MPG by 5 miles per gallon.

In addition, regression uses lots of other evaluation metrics, many of which have built-in penalization for overfitting (and thus don't require cross-validation). Examples include the Akaikie information criterion (AIC) and Bayesian information criterion (BIC). Most textbooks on regression analysis cover these and more advanced topics.

4.3.2 Examining residuals

In the previous section, you saw how the residuals, the distance between the predicted and actual values, were used for both of the simple metrics introduced. These residuals can also be interesting to analyze visually themselves.

Figure 4.24 shows an example residual plot for our MPG dataset. This presents the same information as in the scatter plot in figure 4.23, but zoomed in on the scale of the residuals. In an ideal case, you expect the residuals to be distributed randomly around the 0-line. In the lower end of the figure, MPG values from 10 to 35, it looks like the residuals are randomly distributed around the 0-line, maybe with a slight bias toward overestimating the values. At values 35–45, however, you can see a clear bias

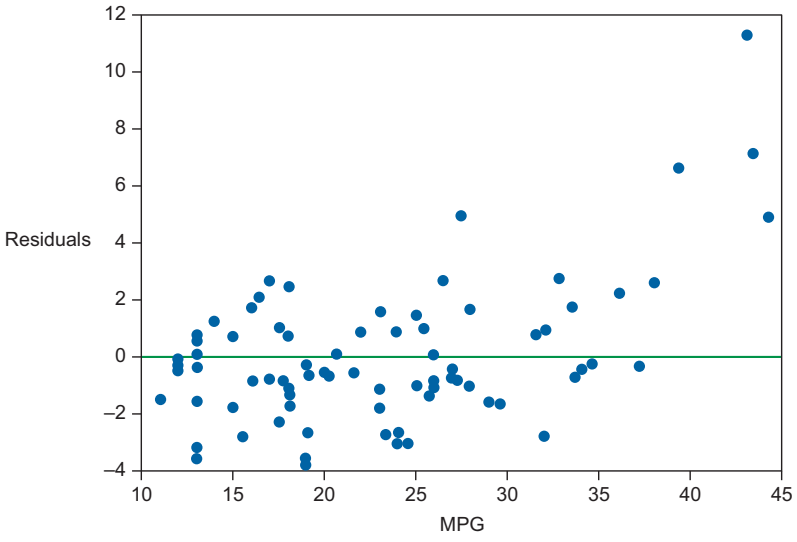


Figure 4.24 The residual plot from predictions on the MPG dataset. At the horizontal 0-line, the residual is 0.

toward underestimating the values, resulting in larger residual values. You can use this information to improve the model, either by tweaking model parameters, or by processing or amending the data. If you can acquire additional data, you could try to obtain labels for a few more high-MPG examples. In this case, you could find a few more high-MPG cars and add them to the dataset in order to improve the predictions in that part of the scale.

You've seen how cross-validation is used to test models and some of the performance metrics you can use to evaluate the results. For the simplest models, this is a matter of training, testing, and computing the appropriate performance metric(s). More-sophisticated algorithms have tuning parameters—knobs that can be turned by the user—that affect how they're trained and applied. Each combination of settings yields a different mode. In the next section, you'll see how sometimes a small adjustment can make a big difference in the results.

4.4 Model optimization through parameter tuning

Most machine-learning models come endowed with one or more *tuning parameters* that control the inner workings of the learning algorithm. These tuning parameters typically control the complexity of the relationship between the input features and target variable. As a result, the tuning parameters can have a strong influence on the fitted model and its predictive accuracy on new data.

For example, in section 4.1 you saw how a single turning parameter (the bandwidth in a kernel-smoothing regression algorithm) can cause wildly different model fits in the corn production dataset. For small values of the bandwidth parameter, the regression function was overly bumpy and overfit the data. Likewise, for large values of the bandwidth parameter, the regression function was too smooth and underfit the data.

This section introduces a rigorous methodology to optimize ML models with respect to the machine-learning algorithm tuning parameters.

4.4.1 ML algorithms and their tuning parameters

Each machine-learning algorithm contains a different set of tuning parameters that control how the algorithm uses training data to build a model. As the algorithms become more sophisticated, typically the tuning parameters become more numerous and esoteric. Here are the standard tuning parameters for some of the popular classification algorithms that you learned about in chapter 3, listed in order of increasing complexity:

- *Logistic regression*—None
- *K-nearest neighbors*—Number of nearest neighbors to average
- *Decision trees*—Splitting criterion, max depth of tree, minimum samples needed to make a split
- *Kernel SVM*—Kernel type, kernel coefficient, penalty parameter

- *Random forest*—Number of trees, number of features to split in each node, splitting criterion, minimum samples needed to make a split
- *Boosting*—Number of trees, learning rate, max depth of tree, splitting criterion, minimum samples needed to make a split

As an example, think back to chapter 3, where you applied a kernel SVM to the Titanic Passengers dataset. You saw that the model fit for two choices of the kernel coefficient parameter (called *gamma*) in figures 3.8 and 3.9. Note that the fits are different: setting gamma = 0.01 produces a complex, segmented decision boundary between the two classes, whereas setting gamma = 0.1 creates a smoother model. In this case, the fitted model is highly sensitive to the choice of the tuning parameter gamma.

What makes this difficult is that the appropriate choice for each tuning parameter for a given algorithm is entirely dependent on the problem and data at hand. What works well for one problem isn't necessarily appropriate for the next problem. Relying on heuristics and rule-of-thumb default tuning parameter settings may lead to poor predictive performance. Rigorous selection of tuning parameters is critical to ensure that your models are as accurate as they can be with the given data.

4.4.2 Grid search

The standard way to optimize the choice of tuning parameters for an ML model is via a brute-force *grid search*. As you map out the following basic grid-search algorithm, note that this strategy ties together the material on cross-validation and model evaluation from the previous sections of this chapter. The grid search algorithm is as follows:

1. Choose the evaluation metric that you want to maximize (for example, AUC for classification, R^2 for regression).
2. Choose which ML algorithm you want to use (for example, random forest).
3. Select which tuning parameters you want to optimize over (for example, number of trees and number of features per split) and the array of values to test for each parameter.
4. Define the grid as the Cartesian product between the arrays of each tuning parameter. For example, if the arrays are [50, 100, 1000] for number of trees and [10, 15] for number of features per split, then the grid is [(50,10), (50,15), (100,10), (100,15), (1000,10), (1000,15)].
5. For each combination of tuning parameters in the grid, use the training set to perform cross-validation (using either the hold-out or k-fold-CV method) and compute the evaluation metric on the cross-validated predictions.
6. Finally, select the set of tuning parameters corresponding to the largest value of the evaluation metric. This is the optimized model.

Why does this work? Grid search does an extensive search over the possible combinations of values for each of the tuning parameters. For each combination, it estimates the performance of that model on new data by comparing the cross-validated predictions to the true target variable. Then, the model with the best estimated accuracy (for

new data) is chosen. This model has the highest likelihood of performing the best when applied to new data.

Let's apply grid search to the Titanic Passengers dataset. You'll use AUC as your optimization metric and SVM with a radial basis function (RBF) kernel as your classification algorithm. You can, in principle, also use grid search to select the best kernel. Indeed, you could use grid search to select between different algorithms!

Next, you select which tuning parameters to optimize over. For kernel SVM with an RBF kernel, you have two standard tuning parameters: the kernel coefficient, gamma; and the penalty parameter, C. The following listing shows how to run a grid search over those two parameters for this problem.

Listing 4.7 Grid search with kernel SVM

```
# Inputs: X – features, y - target

import numpy as np
from sklearn.metrics import roc_auc_score
from sklearn.svm import SVC

# grid of (gamma, C) values to try
gam_vec, cost_vec = np.meshgrid(np.linspace(0.01, 10., 11),
                     np.linspace(1., 10., 11))

AUC_all = []                                    <- Initializes empty array to store AUC results

# set up cross-validation folds
N = len(y)
K = 10                                          <- Number of cross-validation folds
folds = np.random.randint(0, K, size=N)

# search over every value of the grid
for param_ind in np.arange(len(gam_vec.ravel())):

    # initialize cross-validation predictions
    y_cv_pred = np.empty(N)

    # loop through the cross-validation folds
    for ii in np.arange(K):
        # break your data into training and testing subsets
        X_train = X.ix[folds != ii,:]
        y_train = y.ix[folds != ii]
        X_test = X.ix[folds == ii,:]

        # build a model on the training set
        model = SVC(gamma=gam_vec.ravel()[param_ind],
     C=cost_vec.ravel()[param_ind])
        model.fit(X_train, y_train)

        # generate and store model predictions on the testing set
        y_cv_pred[folds == ii] = model.predict(X_test)

    # evaluate the AUC of the predictions
    AUC_all.append(roc_auc_score(y, y_cv_pred))
```

```
indmax = np.argmax(AUC_all)
print "Maximum = %.3f" % (np.max(AUC_all))
print "Tuning Parameters: (gamma = %f, C = %f)" % (gam_vec.ravel()[indmax],
cost_vec.ravel()[indmax])
```

You find with the Titanic dataset that the maximum cross-validated AUC is 0.670, and it occurs at the tuning parameter vector (gamma = 0.01, C = 6). A contour plot showing the AUC evaluated over the grid as in figure 4.25 can be informative. A few factors jump out from this plot:

- The maximum occurs at the boundary of the grid (gamma = 0.01), meaning that you'd want to rerun the grid search on an expanded grid.
- A high amount of sensitivity exists in the accuracy of the predictions to the numerical value of the gamma parameter, meaning that you need to increase the granularity of sampling of that parameter.
- The maximum value occurs near gamma = 0, so expressing the grid on a log scale (for example, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}) is sensible.
- There's not much sensitivity of the AUC as a function of C, so you can use a coarse sampling of that parameter.

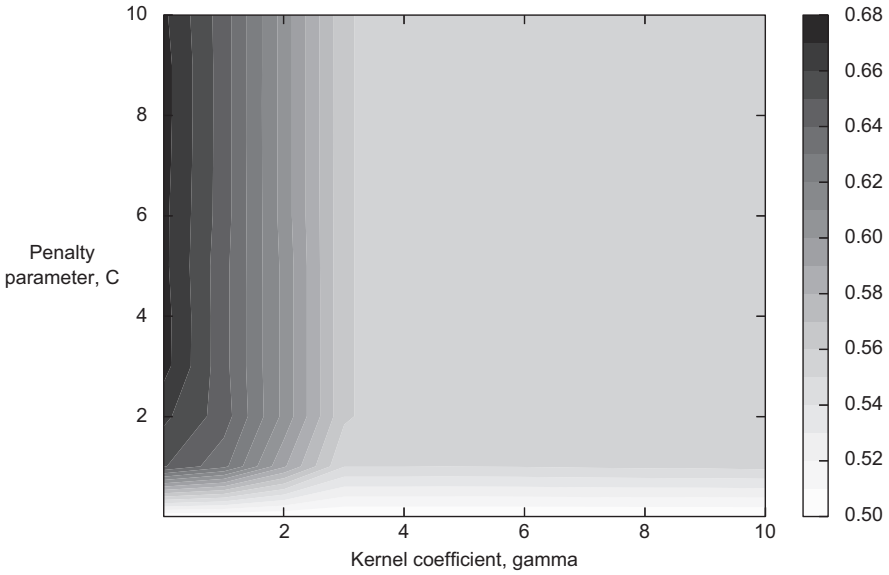


Figure 4.25 Contour plot showing the cross-validated AUC as a function of the two tuning parameters, gamma and C. The maximum occurs way off to the upper left, meaning that you need to expand the search and focus on that region.

Rerunning the grid search on a modified grid, you find that the maximum AUC is 0.690 and it occurs at (gamma = 0.08, C = 20). The value of optimizing over the tuning parameters is clear: a single model with arbitrary choice of tuning parameter could

have attained a result as poor as AUC = 0.5 (no better than random guessing); the grid-search optimized model boosts the accuracy up to AUC = 0.69.

Note that grid search doesn't absolutely ensure that you've chosen the best set of tuning parameters. Because of the limitations caused by choosing a finite grid of possible values to try, the actual best value might have landed somewhere between the values of the grid. Readers who have some familiarity with optimization might be wondering why more-sophisticated optimization routines aren't traditionally used for tuning-parameter selection. The short answer is that the world of derivative-free, nonconvex optimization hasn't yet become part of the standard ML toolkit. The longer answer is that ML researchers on the cutting edge of the field are beginning to incorporate these methods into tuning-parameter optimization strategies.

4.5 Summary

In this chapter, you learned the basics of evaluating ML model performance. Here's a quick rundown of the main takeaways:

- When you evaluate models, you can't double-dip the training data and use it for evaluation as well as training.
- Cross-validation is a more robust method of model evaluation.
- Holdout cross-validation is the simplest form of cross-validation. A testing set is held out for prediction, in order to better estimate the model's capability to be generalized.
- In k-fold cross-validation, k-folds are held out one at a time, providing more-confident estimates of model performance. This improvement comes at a higher computational cost. If available, the best estimate is obtained if k = number of samples, also known as leave-one-out cross-validation.
- The basic model-evaluation workflow is as follows:
  1. Acquire and preprocess the dataset for modeling (chapter 2) and determine the appropriate ML method and algorithm (chapter 3).
  2. Build models and make predictions by using either the holdout or k-fold cross-validation methods, depending on the computing resources available.
  3. Evaluate the predictions with the performance metric of choice, depending on whether the ML method is classification or regression.
  4. Tweak the data and model until the desired model performance is obtained. In chapters 5–8, you'll see various methods for increasing the model performance in common real-world scenarios.
- For classification models, we introduced a few model-performance metrics to be used in step 3 of the workflow. These techniques include simple counting accuracy, the confusion matrix, receiver-operator characteristics, the ROC curve, and the area under the ROC curve.

- For regression models, we introduced the root-mean-square error and R-squared estimators. Simple visualizations, such as the prediction-versus-actual scatter plot and the residual plot, are useful.
- You can use a grid-search algorithm to optimize a model with respect to tuning parameters.

4.6 Terms from this chapter

Word	Definition
underfitting/overfitting	Using a model that's too simple or too complex, respectively, for the problem at hand.
evaluation metric	A number that characterizes the performance of the model.
mean squared error	A specific evaluation metric used in regression models.
cross-validation	The method of splitting the training set into two or more training/testing sets in order to better assess the accuracy.
holdout method	A form of cross-validation in which a single test set is held out of the model-fitting routine for testing purposes.
k-fold cross-validation	A kind of cross-validation in which data is split into k random disjoint sets (folds). The folds are held out one at a time, and cross-validated on models built on the remainder of the data.
confusion matrix	A matrix showing for each class the number of predicted values that were correctly classified or not.
receiver operating characteristic (ROC)	A number representing true positives, false positives, true negatives, or false negatives.
area under the ROC curve (AUC)	An evaluation metric for classification tasks defined from the area under the ROC curve of false positives versus true positives.
tuning parameter	An internal parameter to a machine-learning algorithm, such as the bandwidth parameter for kernel-smoothing regression.
grid search	A brute-force strategy for selecting the best values for the tuning parameters to optimize an ML model.

In the next chapter, you'll start looking at improving your models by focusing on their features. In addition to basic feature-engineering techniques, you'll learn advanced methods for extracting information out of text, images, and time-series data. You'll also see how to select the best features to optimize the performance of the model and avoid overfitting.